Name: Nikhil Garg **Roll Number:** MT2025076 **Course:** CSE-816

Minor Project: Scientific Calculator with DevOps Toolchain

1 What and Why of DevOps?

DevOps is a set of practices that combines software development (Dev) and IT operations (Ops). It aims to shorten the systems development life cycle and provide continuous delivery with high software quality.

- **Speed:** Rapid delivery of features and fixes.
- **Automation:** Reducing manual errors in testing and deployment through CI/CD pipelines.
- **Consistency:** Ensuring the application runs identically across all environments using Docker containerization.

2 DevOps Pipeline Overview

The implemented DevOps pipeline automates the entire software production lifecycle, from initial code commit to final deployment on a managed host[cite: 324].

Developer → GitHub → Jenkins → JUnit → Maven → Docker → DockerHub → Ansible

This seamless integration ensures high software quality and continuous delivery. The workflow is characterized by the following automated triggers and monitoring mechanisms:

- **Automated Triggering:** The pipeline is initiated automatically through GitHub webhooks, which are exposed to the public internet using an **ngrok** tunnel.
- **Continuous Monitoring:** Jenkins utilizes GitSCM polling to monitor the repository and triggers the pipeline immediately upon detecting code changes.
- **Feedback Loop:** The system provides immediate feedback through SMTP email notifications if any stage of the build or test process fails.

Through this architecture, the project achieves rapid delivery of features while maintaining environmental consistency via containerization.

3 Tools Used

The following DevOps tool chain was implemented to satisfy the project requirements:

Stage	Tool	Description
Source Control	GitHub	Central repository for version control.
Automation Tunnel	ngrok	Public URL to expose local Jenkins for Webhooks.
Notification	SMTP (Email)	Automated alerts for build failures.
Continuous Integration	Jenkins	Pipeline orchestration.
Testing	JUnit	Automated unit testing for calculator logic.
Build Tool	Maven	Project dependency and build management.
Containerization	Docker	Creating portable images.
Deployment	Ansible	Automated deployment on managed hosts.

4 Implementation Steps

4.1 Step 1: Source Control Management (GitHub)

A repository was created to host the source code, tests, and DevOps configuration files.

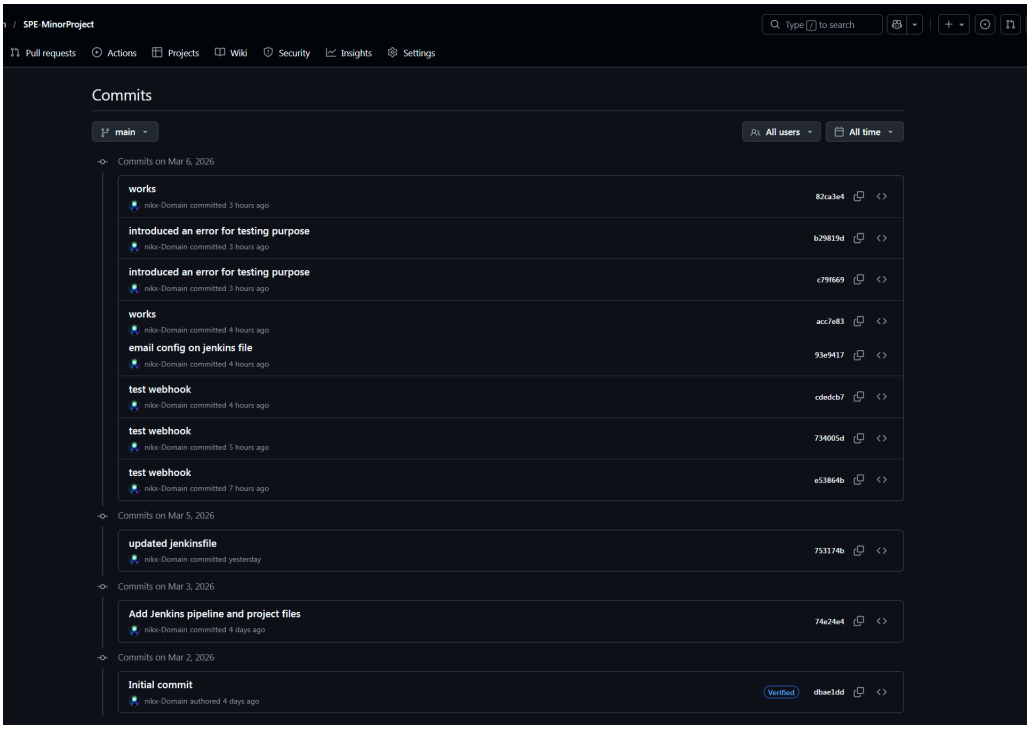


Figure 1: GitHub Commit History and Repository Structure

4.2 Step 2: Automated Trigger (ngrok & Webhooks)

To enable automated builds, **ngrok** was used to tunnel the local Jenkins instance to the internet, allowing GitHub to send **Webhooks** on every push.

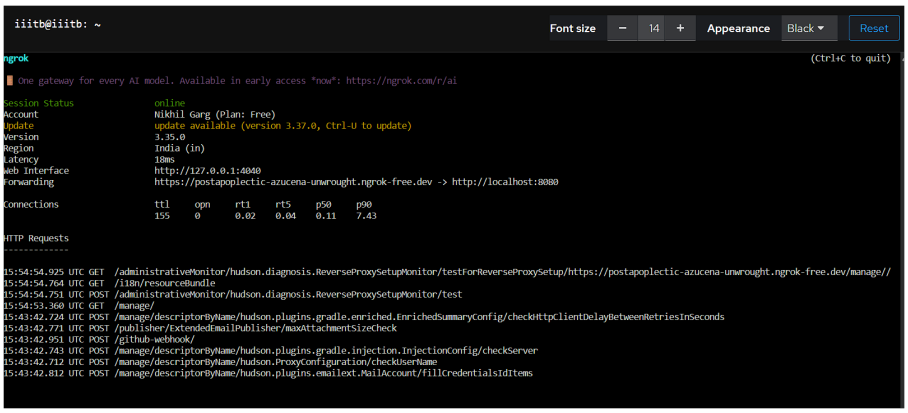


Figure 2: Active ngrok session establishing a secure gateway for GitHub Webhook integration.

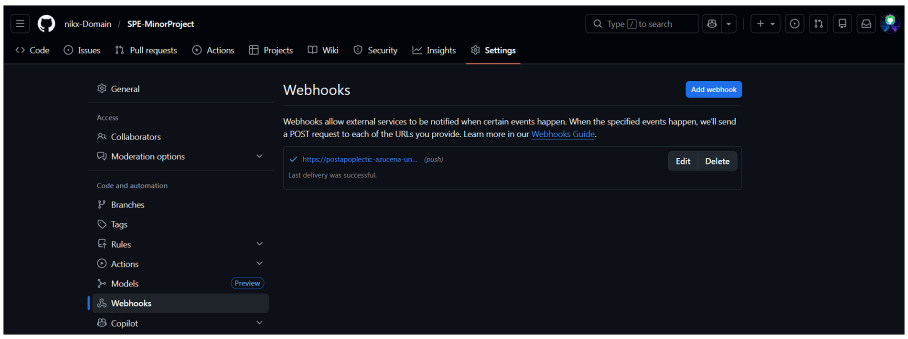


Figure 3: GitHub Webhook configuration using the generated ngrok URL

4.3 Step 3: Build and Test (Maven & JUnit)

The scientific calculator supports Square Root, Factorial, Natural Log, and Power. Maven handles the build, while JUnit ensures the accuracy of these mathematical operations. The build stage compiles the Java source code and executes unit tests using the `mvn clean package` command. This process ensures the application is built correctly from a fresh state:

- **Clean Phase:** The `clean` command removes the `target/` directory, which contains all previously generated build files and compiled classes.
- **Fresh Environment:** By removing old artifacts, it ensures that the current build is not influenced by stale data or residual files from previous executions.
- **Compile Stage:** Maven converts the Java source code (`ScientificCalculator.java`) into compiled bytecode files.
- **Dependency Management:** It automatically identifies and downloads any project dependencies specified in the `pom.xml`.

- **Test Execution:** Before packaging, Maven automatically executes all JUnit test cases defined in the project to verify the mathematical logic of the calculator.
- **Artifact Generation:** Once tests pass successfully, the `package` command bundles the compiled code into an executable JAR file.
- **Pipeline Success:** This stage serves as a gatekeeper; if compilation fails or any unit test fails, Jenkins stops the pipeline and triggers a failure notification.

```
$ mvn clean package
```

Figure 4: Maven test results showing successful JUnit execution and BUILD SUCCESS.

4.4 Step 4: Continuous Integration (Jenkins)

A Jenkins Pipeline was configured to automate the entire lifecycle. The system is specifically configured to send an **Email Notification** via SMTP to the developer if any stage fails.

Figure 5: Jenkins Pipeline Stage View showing successful automation.

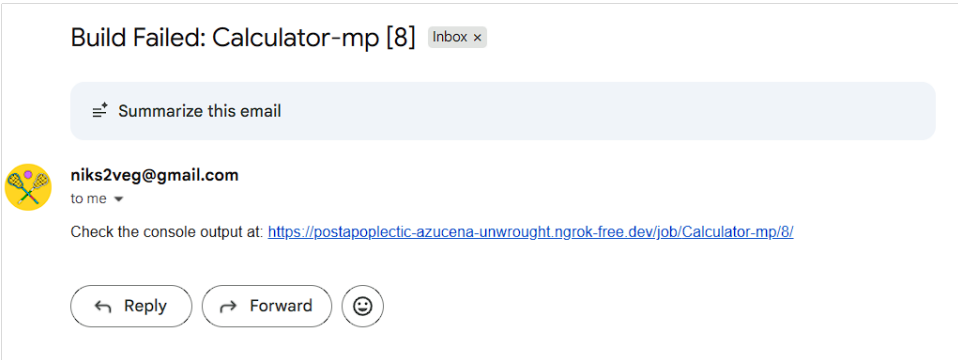


Figure 6: Automated SMTP Email Notification received upon a build failure.

4.5 Step 5: Containerization (Docker)

The application was packaged into a lightweight Docker image using a multi-stage **Dockerfile** and pushed to **Docker Hub**.

```
$ docker push nikxdomain/scientific-calculator
```

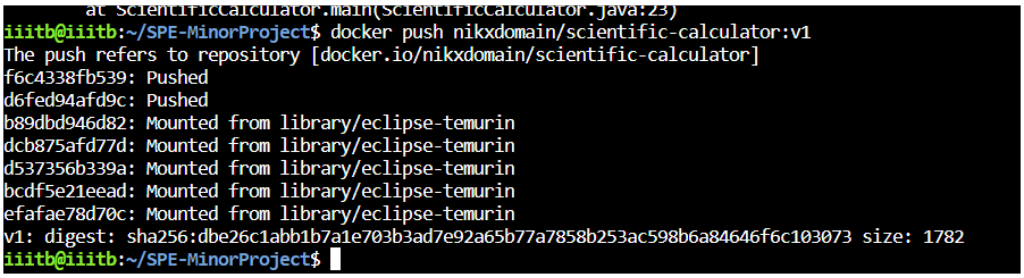


Figure 7: Terminal output: Pushing the image onto Docker Hub.

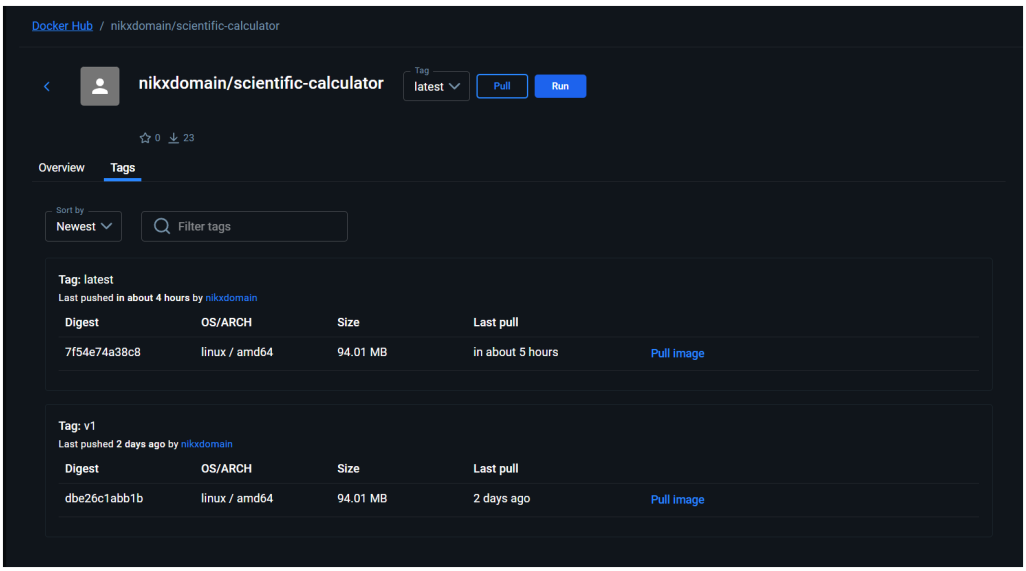
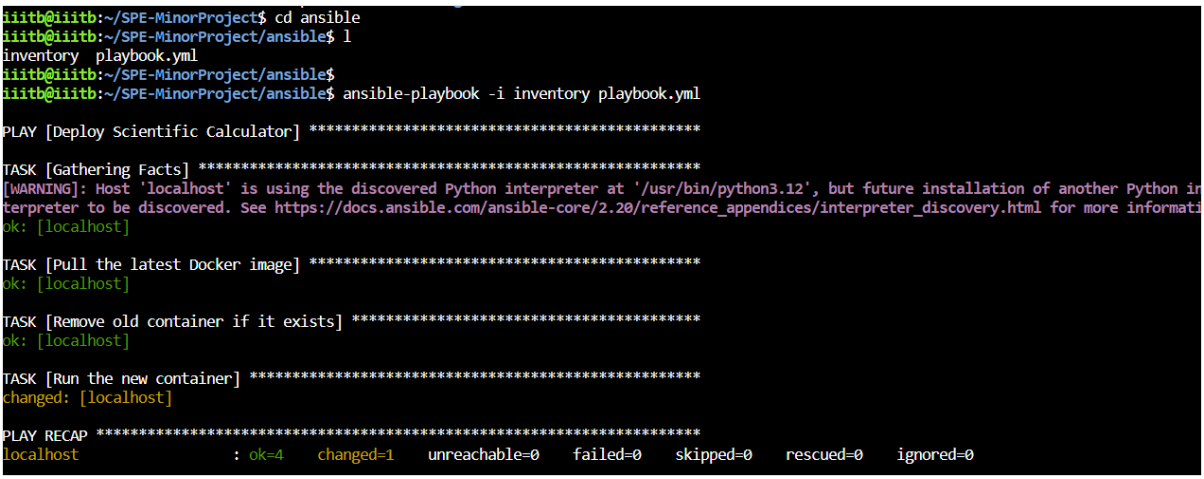


Figure 8: Docker Hub Repository showing the successfully pushed `latest` image.

4.6 Step 6: Deployment (Ansible)

Ansible was used to pull the verified image from Docker Hub and deploy it on the managed host, ensuring the latest version is always running.



```
iiitb@iiitb:~/SPE-MinorProject$ cd ansible
iiitb@iiitb:~/SPE-MinorProject/ansible$ l
inventory  playbook.yml
iiitb@iiitb:~/SPE-MinorProject/ansible$
iiitb@iiitb:~/SPE-MinorProject/ansible$ ansible-playbook -i inventory playbook.yml

PLAY [Deploy Scientific Calculator] ********************************************

TASK [Gathering Facts] *********************************************************
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python in
terpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more informati
ok: [localhost]

TASK [Pull the latest Docker image] ********************************************
ok: [localhost]

TASK [Remove old container if it exists] ***************************************
ok: [localhost]

TASK [Run the new container] ***************************************************
changed: [localhost]

PLAY RECAP *********************************************************************
localhost                  : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Figure 9: Terminal output showing successful Ansible Playbook execution recap.

5 Scripts

5.1 Jenkinsfile

```
pipeline {
    agent any
    tools {
        maven 'maven3'
    }

    environment {
        DOCKER_HUB_CREDENTIALS = credentials('dockerhub-credentials-id')
        IMAGE_NAME = 'nikxdomain/scientific-calculator'
        IMAGE_TAG = 'latest'
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build and Test') {
            steps {
                sh 'mvn clean package'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh "docker build -t ${IMAGE_NAME}:${IMAGE_TAG} ."
            }
        }

        stage('Push Docker Image') {
            steps {
                sh "echo \$DOCKER_HUB_CREDENTIALS_PSW | docker login -u \$DOCKER_HUB_CREDENTIALS_USR --password-stdin"
                sh "docker push ${IMAGE_NAME}:${IMAGE_TAG}"
            }
        }
    }

    post {
        failure {
            mail to: 'niks2veg@gmail.com',
                 subject: "Build Failed: ${env.JOB_NAME} [${env.BUILD_NUMBER}]",
                 body: "Check the console output at: ${env.BUILD_URL}"
        }
        success {
            echo 'Build succeeded! No email sent.'
        }
    }
}
```

5.2 Dockerfile

```
FROM eclipse-temurin:11-jre
WORKDIR /app
COPY target/scientific-calculator-1.0-SNAPSHOT.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

5.3 Ansible Playbook

```
---
- name: Deploy Scientific Calculator
  hosts: managed_hosts
  tasks:
    - name: Pull the latest Docker image
      docker_image:
        name: nikxdomain/scientific-calculator:latest
        source: pull

    - name: Remove old container if it exists
      docker_container:
        name: calculator_app
        state: absent

    - name: Run the new container
      docker_container:
        name: calculator_app
        image: nikxdomain/scientific-calculator:latest
        state: started
        interactive: yes
        tty: yes
```

6 Project Links

- **GitHub Repository:** `https://github.com/nikx-Domain/SPE-MinorProject`
- **Docker Hub:** `https://hub.docker.com/r/nikxdomain/scientific-calculator`